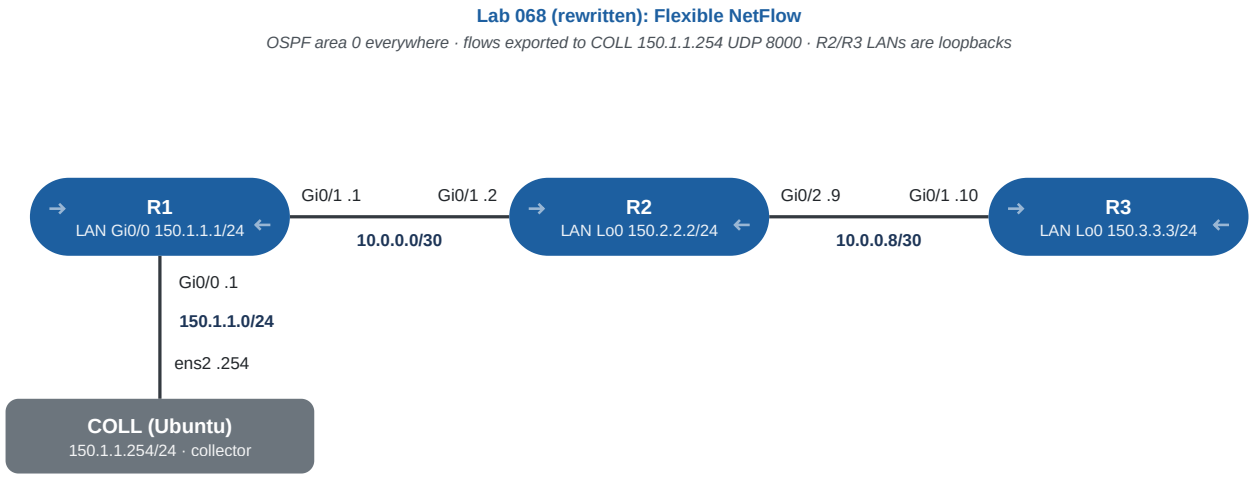
# CCNP Lab 068: Flexible NetFlow Lab (rewritten)

## Lab Objective:

The focus of this lab is to understand Flexible NetFlow (FNF) implementation and configuration in Cisco IOS routers. Additional technologies include custom flow records, local-only flow caches, flow exporters and NetFlow v9 export to a collector.

## Lab Topology:

The lab network topology is illustrated below:




## IMPORTANT NOTE

This is the book's Lab 068 (NetFlow and Accounting) rewritten for **Flexible NetFlow**, which is what ENCOR tests. The tasks follow the original ones: IP accounting becomes a local FNF monitor, and the traditional `ip flow` / `ip flow-export` commands become flow records, exporters and monitors.

Use IOSv for R1, R2 and R3, and an Ubuntu node for COLL. The serial links are replaced by GigabitEthernet. The R2 and R3 LANs are loopbacks.

Flexible NetFlow is built from three pieces:

- **flow record**: *what* to track. `match` fields define a flow (key fields), `collect` fields are only recorded (counters, timestamps and so on).
- **flow exporter**: *where* to send the flows (collector, port, source, version).
- **flow monitor**: ties a record and an exporter to a cache. The monitor is what you apply to an interface, with a direction.

## Task 1

Configure hostnames and IP addresses on all routers as illustrated in the topology. Configure COLL with 150.1.1.254/24 and a default route to R1.

## Task 2

Configure OSPF area 0 on all routers for all interfaces. Make the loopbacks appear with their real /24 mask. Verify that COLL can ping 150.3.3.3.

## Task 3

Configure Flexible NetFlow on all router WAN interfaces. All routers should keep packet and byte counts per source and destination IP address for all packets egressing the WAN interfaces. The data should be stored on the router only and not exported. Verify the configuration and generate transit traffic to test it.

## Task 4

Configure Flexible NetFlow on all routers to collect all ingress flows on every GigabitEthernet interface, using the predefined record that matches traditional NetFlow. Export the flows to COLL 150.1.1.254 on UDP port 8000 using NetFlow version 9. The exports should be sourced from the LAN interface of each router (R1 Gi0/0, R2 and R3 Loopback0). Long-lived flows should be exported every 60 seconds. Verify that COLL receives the exports.

## Task 5

Assume that R1 is a core Internet router. On R1's LAN interface, replace the Task 4 monitor with a new one that:

- stores up to 128K (131072) entries in the cache
- records the source and destination autonomous system (AS) of each flow
- records the source MAC address of the incoming frame and the destination MAC address of the outgoing frame
- still exports to COLL

Verify your configuration.

# Lab Validation:

The outputs below are trimmed and were written by hand, not captured, so small details can differ on IOSv.

### Task 1

```
R1(config)#interface GigabitEthernet0/0
R1(config-if)#ip address 150.1.1.1 255.255.255.0
R1(config-if)#no shutdown
R1(config-if)#interface GigabitEthernet0/1
R1(config-if)#ip address 10.0.0.1 255.255.255.252
R1(config-if)#no shutdown
```

```
R2(config)#interface Loopback0
R2(config-if)#ip address 150.2.2.2 255.255.255.0
R2(config-if)#ip ospf network point-to-point
R2(config-if)#interface GigabitEthernet0/1
R2(config-if)#ip address 10.0.0.2 255.255.255.252
R2(config-if)#no shutdown
R2(config-if)#interface GigabitEthernet0/2
```

```
R2(config-if)#ip address 10.0.0.9 255.255.255.252
R2(config-if)#no shutdown
```

R3 is the same idea: Loopback0 150.3.3.3/24 (with `ip ospf network point-to-point`) and Gi0/1 10.0.0.10/30.

```
cisco@COLL:~$ sudo ip addr add 150.1.1.254/24 dev ens2
cisco@COLL:~$ sudo ip link set ens2 up
cisco@COLL:~$ sudo ip route add default via 150.1.1.1
```

## Task 2

On every router:

```
R1(config)#router ospf 1
R1(config-router)#network 0.0.0.0 255.255.255.255 area 0
```

```
cisco@COLL:~$ ping -c 3 150.3.3.3
64 bytes from 150.3.3.3: icmp_seq=1 ttl=253 time=2.10 ms
```

## Task 3

This replaces the original IP accounting task. You build your own record with only the fields you care about, put it in a monitor **without an exporter** (so the data stays on the router), and apply it in the **output** direction.

```
R2(config)#flow record REC-ACCT
R2(config-flow-record)#match ipv4 source address
R2(config-flow-record)#match ipv4 destination address
R2(config-flow-record)#collect counter bytes
R2(config-flow-record)#collect counter packets
R2(config-flow-record)#exit
R2(config)#flow monitor MON-ACCT
R2(config-flow-monitor)#record REC-ACCT
R2(config-flow-monitor)#exit
R2(config)#interface GigabitEthernet0/1
R2(config-if)#ip flow monitor MON-ACCT output
R2(config-if)#interface GigabitEthernet0/2
R2(config-if)#ip flow monitor MON-ACCT output
```

R1 and R3 get the same record and monitor, applied on their single WAN interface (R1 Gi0/1, R3 Gi0/1).

- `match` = **key fields**: every new combination of source and destination address is a new flow
- `collect` = **non-key fields**: just recorded for each flow

To test, send transit traffic through R2 and check its cache:

```
R1#ping 150.3.3.3 source GigabitEthernet0/0 repeat 100
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
Success rate is 100 percent (100/100)
```

```
R2#show flow monitor MON-ACCT cache format table
  Cache type:                               Normal
  Cache size:                                 4096
  Current entries:                               2

IPV4 SRC ADDR    IPV4 DST ADDR    bytes        pkts
===============  ===============  ===========  ====
```

```
150.1.1.1       150.3.3.3             10000   100
150.3.3.3       150.1.1.1             10000   100
```

R2 sees both directions because the echoes leave on Gi0/2 and the replies leave on Gi0/1. Other useful commands are `show flow record REC-ACCT`, `show flow monitor MON-ACCT` and `show flow interface GigabitEthernet0/1`.

## Task 4

Flexible NetFlow can't export version 5. It uses **version 9** (or IPFIX). The predefined record `netflow ipv4 original-input` uses the same key fields as traditional NetFlow: source and destination IP and ports, protocol, ToS and input interface.

```
R1(config)#flow exporter EXP
R1(config-flow-exporter)#destination 150.1.1.254
R1(config-flow-exporter)#source GigabitEthernet0/0
R1(config-flow-exporter)#transport udp 8000
R1(config-flow-exporter)#export-protocol netflow-v9
R1(config-flow-exporter)#exit
R1(config)#flow monitor MON-IN
R1(config-flow-monitor)#record netflow ipv4 original-input
R1(config-flow-monitor)#exporter EXP
R1(config-flow-monitor)#cache timeout active 60
R1(config-flow-monitor)#exit
R1(config)#interface GigabitEthernet0/0
R1(config-if)#ip flow monitor MON-IN input
R1(config-if)#interface GigabitEthernet0/1
R1(config-if)#ip flow monitor MON-IN input
```

R2 and R3 are the same, with `source Loopback0` in the exporter and the monitor on their Gi interfaces (R2 Gi0/1 and Gi0/2, R3 Gi0/1).

- `netflow-v9` is the default export protocol, so that line is optional
- **Active timeout**: a flow that is still running is exported every 60 seconds (the default is 1800)
- **Inactive timeout**: a flow with no packets for 15 seconds (default) is exported and removed from the cache

Generate some traffic from COLL and check the cache on R3:

```
cisco@COLL:~$ ping -c 5 150.3.3.3
```

```
R3#show flow monitor MON-IN cache format table
IPV4 SRC ADDR    IPV4 DST ADDR    TRNS DST PORT  INTF INPUT  IP PROT  bytes  pkts
===============  ===============  =============  ==========  =======  =====  ====
150.1.1.254      150.3.3.3                 2048  Gi0/1             1    420     5
```

Some columns (source port, ToS, timestamps and so on) are left out above to fit the page.

For ICMP, the destination port field shows the ICMP type × 256 + code, so echo request (type 8) shows as **2048**.

```
R1#show flow exporter EXP statistics
Flow Exporter EXP:
  Packet send statistics (last cleared 00:04:31 ago):
    Successfully sent:         6                     (1380 bytes)
```

```
cisco@COLL:~$ sudo tcpdump -ni ens2 udp port 8000
IP 150.1.1.1.51234 > 150.1.1.254.8000: UDP, length 116
```

```
IP 150.2.2.2.60871 > 150.1.1.254.8000: UDP, length 116
IP 150.3.3.3.55102 > 150.1.1.254.8000: UDP, length 116
```

Seeing the UDP packets is enough proof. COLL has no collector software, so it may answer with ICMP port unreachable, which is harmless.

## Task 5

These features used to be extra `ip flow-*` commands. In Flexible NetFlow they're just **more fields in the record** and a **cache setting in the monitor**:

```
R1(config)#flow record REC-CORE
R1(config-flow-record)#match ipv4 source address
R1(config-flow-record)#match ipv4 destination address
R1(config-flow-record)#match ipv4 protocol
R1(config-flow-record)#match transport source-port
R1(config-flow-record)#match transport destination-port
R1(config-flow-record)#match interface input
R1(config-flow-record)#collect routing source as
R1(config-flow-record)#collect routing destination as
R1(config-flow-record)#collect datalink mac source address input
R1(config-flow-record)#collect datalink mac destination address output
R1(config-flow-record)#collect counter bytes
R1(config-flow-record)#collect counter packets
R1(config-flow-record)#exit
R1(config)#flow monitor MON-CORE
R1(config-flow-monitor)#record REC-CORE
R1(config-flow-monitor)#exporter EXP
R1(config-flow-monitor)#cache entries 131072
R1(config-flow-monitor)#exit
R1(config)#interface GigabitEthernet0/0
R1(config-if)#no ip flow monitor MON-IN input
R1(config-if)#ip flow monitor MON-CORE input
```

The task asks for a **new** monitor because IOS usually won't let you change the record of a monitor that's already applied to an interface. You'd have to remove it from every interface first.

```
R1#show flow monitor MON-CORE
Flow Monitor MON-CORE:
  Description:       User defined
  Flow Record:       REC-CORE
  Flow Exporter:     EXP
  Cache:
    Type:                 normal (Platform cache)
    Status:               allocated
    Size:                 131072 entries
    Inactive Timeout:     15 secs
    Active Timeout:       1800 secs
```

```
R1#show flow monitor MON-CORE cache
IPV4 SOURCE ADDRESS:       150.1.1.254
IPV4 DESTINATION ADDRESS:  150.3.3.3
TRNS SOURCE PORT:          0
TRNS DESTINATION PORT:     2048
INTERFACE INPUT:           Gi0/0
IP PROTOCOL:               1
routing source as:         0
routing destination as:    0
datalink mac source address input:       5254.0011.2233
datalink mac destination address output: 5254.0044.5566
counter bytes:             420
counter packets:           5
```

The AS fields stay **0** here, because AS numbers come from BGP and this lab only runs OSPF. On a real Internet router with BGP they'd show the origin AS. Note that MON-CORE kept the default 1800-second active timeout, because the task didn't ask for 60.

## Traditional NetFlow vs Flexible NetFlow

| Book's original (traditional) | Flexible NetFlow |
|---|---|
| `ip accounting output-packets` | monitor without exporter, applied `output` |
| `ip flow ingress` | `ip flow monitor <name> input` |
| `ip flow-export destination <ip> <port>` | `flow exporter` → `destination` + `transport udp <port>` |
| `ip flow-export version 5` | `export-protocol netflow-v9` (no v5 in FNF) |
| `ip flow-export source <int>` | `flow exporter` → `source <int>` |
| `ip flow-cache entries <n>` | `flow monitor` → `cache entries <n>` |
| `ip flow-export version 5 origin-as` | record → `collect routing source/destination as` |
| `ip flow-capture mac-addresses` | record → `collect datalink mac ...` |
| `show ip cache flow` | `show flow monitor <name> cache` |